

Phase 3: Closed Hashing with Linear Probing — Full Structured Note

1. Current Progress Analysis

Before Phase 3, you already studied two important hashing ideas.

In **Phase 1**, you learned the foundation of hashing:

Key → Hash Function → Index

Example:

$H(15) = 15 \bmod 10 = 5$

This means key **15** should be placed at index **5**.

You also learned that collisions can happen.

Example:

$15 \bmod 10 = 5$
 $25 \bmod 10 = 5$

Both keys want the same index. This is called a **collision**.

In **Phase 2**, you learned the first collision-resolution method: **open hashing with chaining**.

In chaining, if many keys go to the same index, we store them in a linked list.

Example:

Index 5: 5 → 25 → 75 → NULL

Now Phase 3 teaches a different method.

Instead of using linked lists, we keep every key inside the original hash table array.

This is called:

Closed Hashing with Linear Probing

The main idea is:

If the correct index is full, move forward until you find an empty slot.

2. Main Goal of Phase 3

The main goal of Phase 3 is to understand how to handle collisions **inside the same array**.

In this phase, you learn:

- What closed hashing means
- What open addressing means
- What linear probing means
- The linear probing formula
- How wrap-around works
- How insertion works in linear probing
- How searching works in linear probing
- Why search stops at an empty slot
- What loading factor means in linear probing
- Why $\lambda \leq 0.5$ is recommended
- What primary clustering means
- Why deletion is difficult in linear probing
- What rehashing means
- What tombstones mean
- How linear probing is implemented in C
- Common beginner mistakes

The main beginner idea is:

Collision does not create a linked list.
Collision makes us search for another empty slot inside the same table.

3. What Problem Does Phase 3 Solve?

Suppose we have:

Table size = 10
Hash function $H(x) = x \bmod 10$

Now insert:

15, 25

Calculate their indices:

$15 \bmod 10 = 5$
 $25 \bmod 10 = 5$

Both keys want index 5.

In chaining, we would do this:

Index 5: 15 -> 25 -> NULL

But in Phase 3, we do not use a linked list.

Instead, we keep the keys inside the array.

Example:

Index 5: 15
Index 6: 25

Key 25 wanted index 5, but index 5 was full. So it moved to the next empty slot.

This is the basic idea of **linear probing**.

4. Closed Hashing

4.1 What closed hashing means

Closed hashing means all keys must be stored inside the original hash table array.

No linked lists are used.

No outside chains are created.

If a key hashes to a full index, the algorithm must find another empty index inside the same array.

Simple meaning:

Closed hashing = keep everything inside the table.

4.2 Closed hashing compared with chaining

Method	What happens during collision?	Extra linked-list memory?
Open hashing / Chaining	Put colliding keys in a linked list	Yes
Closed hashing / Open addressing	Find another empty slot in the array	No

In Phase 2, collisions became chains.

In Phase 3, collisions become a search for another empty cell.

5. Open Addressing

Closed hashing is also called **open addressing**.

This name can feel confusing at first.

It means:

If the original address is occupied, search for another open address.

Example:

25 wants index 5
Index 5 is full
Try index 6
If index 6 is full, try index 7
Continue until an empty index is found

Simple meaning:

Open addressing = find another open place inside the table.

6. Linear Probing

6.1 What linear probing means

Linear probing is a collision-resolution method used in closed hashing.

It means:

If the index is full, check the next index.

If that is also full, check the next one again.

So we move linearly, one position at a time.

Example:

```
Try index 5  
If full, try index 6  
If full, try index 7  
If full, try index 8
```

This is why it is called **linear** probing.

6.2 Real-life analogy

Imagine a parking lot.

A car wants to park in space number **5**.

If space **5** is empty, the car parks there.

If space **5** is full, the car checks space **6**.

If space **6** is full, it checks space **7**.

It keeps moving forward until it finds an empty space.

That is linear probing.

Hashing idea:

Parking example	Hashing idea
Car	Key
Parking space	Hash table index
Preferred parking space	Original hash index
Full parking space	Collision
Next parking space	Next probe

7. Linear Probing Formula

The general probing formula is:

$$H'(x) = (H(x) + f(i)) \bmod \text{size}$$

For linear probing:

$$f(i) = i$$

So the formula becomes:

$$H'(x) = (H(x) + i) \bmod \text{size}$$

Where:

Symbol	Meaning
x	The key
$H(x)$	Original hash index
i	Probe number
size	Hash table size
$\bmod \text{size}$	Keeps the index inside the table

8. Meaning of `i`

In linear probing:

`i = 0, 1, 2, 3, ...`

Each value of `i` gives a new place to check.

Probe	Formula	Meaning
<code>i = 0</code>	<code>(H(x) + 0) mod size</code>	Try original index
<code>i = 1</code>	<code>(H(x) + 1) mod size</code>	Try next index
<code>i = 2</code>	<code>(H(x) + 2) mod size</code>	Try next next index
<code>i = 3</code>	<code>(H(x) + 3) mod size</code>	Keep moving forward

Important point:

`i` starts at 0 because the first try is the original hash index.

9. Wrap-Around

9.1 What wrap-around means

Wrap-around means when probing reaches the end of the table, it goes back to the beginning.

Example with table size `10`:

Valid indices: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9

If we are at index `9` and need to move forward, we go to index `0`.

After index 9, go back to index 0

9.2 Why modulus gives wrap-around

The formula uses:

mod size

Example:

$$(9 + 1) \bmod 10 = 10 \bmod 10 = 0$$

So after index 9, the next index becomes 0.

Another example:

$$(9 + 2) \bmod 10 = 11 \bmod 10 = 1$$

So two steps after index 9 gives index 1.

Simple meaning:

Modulus prevents the algorithm from going outside the array.

10. Insertion in Linear Probing

Insertion means adding a key to the hash table.

Steps:

1. Take the key.
2. Calculate the original hash index using $H(x)$.
3. If that index is empty, store the key there.
4. If that index is full, move to the next index.
5. Continue until an empty slot is found.
6. Use mod size so the search wraps around if needed.

Simple rule:

Insert at the first empty slot found in the probe sequence.

11. Full Insertion Dry Run

Use:

```
Table size = 10  
Hash function  $H(x) = x \bmod 10$   
Keys = 26, 30, 45, 23, 25, 43, 74, 19, 29
```

At the start, the table is empty.

Index:	0	1	2	3	4	5	6	7	8	9
Value:	_	_	_	_	_	_	_	_	_	_

11.1 Insert 26

Calculate:

$26 \bmod 10 = 6$

Index is empty.

Store at index .

Index:	0	1	2	3	4	5	6	7	8	9
Value:	_	_	_	_	_	_	26	_	_	_

11.2 Insert 30

Calculate:

$30 \bmod 10 = 0$

Index is empty.

Store at index .

Index:	0	1	2	3	4	5	6	7	8	9
Value:	30	_	_	_	_	_	26	_	_	_

11.3 Insert 45

Calculate:

$45 \bmod 10 = 5$

Index is empty.

Store at index .

Index:	0	1	2	3	4	5	6	7	8	9
Value:	30	_	_	_	_	45	26	_	_	_

11.4 Insert 23

Calculate:

$23 \bmod 10 = 3$

Index is empty.

Store at index .

Index:	0	1	2	3	4	5	6	7	8	9
Value:	30	_	_	23	_	45	26	_	_	_

11.5 Insert 25

Calculate:

$$25 \bmod 10 = 5$$

Index 5 already has 45.

This is a collision.

Use linear probing:

Index 5 is full
Index 6 is full
Index 7 is empty

Store 25 at index 7.

Index:	0	1	2	3	4	5	6	7	8	9
Value:	30	_	_	23	_	45	26	25	_	_

11.6 Insert 43

Calculate:

$$43 \bmod 10 = 3$$

Index 3 already has 23.

This is a collision.

Use linear probing:

Index 3 is full
Index 4 is empty

Store 43 at index 4.

Index:	0	1	2	3	4	5	6	7	8	9
Value:	30	_	_	23	43	45	26	25	_	_

11.7 Insert 74

Calculate:

$$74 \bmod 10 = 4$$

Index **4** already has **43**.

This is a collision.

Use linear probing:

```
Index 4 is full
Index 5 is full
Index 6 is full
Index 7 is full
Index 8 is empty
```

Store **74** at index **8**.

Index:	0	1	2	3	4	5	6	7	8	9
Value:	30	_	_	23	43	45	26	25	74	_

11.8 Insert 19

Calculate:

$$19 \bmod 10 = 9$$

Index **9** is empty.

Store **19** at index **9**.

Index:	0	1	2	3	4	5	6	7	8	9
Value:	30	_	_	23	43	45	26	25	74	19

11.9 Insert 29

Calculate:

$$29 \bmod 10 = 9$$

Index 9 already has 19.

This is a collision.

Use linear probing:

Index 9 is full
Move forward with wrap-around
Index 0 is full
Index 1 is empty

Store 29 at index 1.

Index:	0	1	2	3	4	5	6	7	8	9
Value:	30	29	_	23	43	45	26	25	74	19

12. Insertion Dry Run Table

Key	Initial hash	Probe path	Stored at
26	6	Index 6 free	6
30	0	Index 0 free	0
45	5	Index 5 free	5
23	3	Index 3 free	3
25	5	5 full, 6 full, 7 free	7
43	3	3 full, 4 free	4
74	4	4 full, 5 full, 6 full, 7 full, 8 free	8
19	9	Index 9 free	9
29	9	9 full, 0 full, 1 free	1

Final table:

Index	Value
0	30
1	29
2	empty
3	23
4	43
5	45
6	26
7	25
8	74
9	19

13. Important Observation After Insertion

Look at the final table:

```
Index 3: 23
Index 4: 43
Index 5: 45
Index 6: 26
Index 7: 25
Index 8: 74
Index 9: 19
```

There is a long continuous block from index **3** to index **9**.

This block is called a **cluster**.

This is important because linear probing can create long crowded areas in the table.

This problem is called **primary clustering**.

14. Searching in Linear Probing

Searching means checking whether a key exists in the hash table.

Search steps:

1. Take the search key.
2. Calculate the original hash index.
3. Check that index.
4. If the key is found, stop successfully.
5. If the slot has another key, move to the next slot.
6. Continue using the same probe sequence.
7. If an empty slot is found, stop and say the key is not present.

Important:

Search follows the same path that insertion would follow.

15. Why Search Stops at an Empty Slot

In linear probing, insertion always uses the first empty slot it finds.

So if searching reaches an empty slot, the key cannot be after that empty slot.

Why?

Because if the key had been inserted earlier, it would have stopped at that empty slot.

Simple meaning:

A key cannot jump over an empty slot during insertion.

Therefore:

During search, EMPTY means stop.

16. Search Dry Run

Use the final table:

Index:	0	1	2	3	4	5	6	7	8	9
Value:	30	29	empty	23	43	45	26	25	74	19

16.1 Search for 45

Calculate:

$45 \bmod 10 = 5$

Check index **5**:

Index 5 contains 45

Result:

45 found immediately

16.2 Search for 74

Calculate:

$74 \bmod 10 = 4$

Check from index **4**:

Index 4: 43, not 74
Index 5: 45, not 74
Index 6: 26, not 74
Index 7: 25, not 74
Index 8: 74, found

Result:

74 found after 5 checks

This shows a weakness of linear probing.

Even though hashing is supposed to be fast, clustering can make search check many positions.

16.3 Search for 40

Calculate:

$40 \bmod 10 = 0$

Check from index 0:

Index 0: 30, not 40
Index 1: 29, not 40
Index 2: empty

Stop at index 2 because it is empty.

Result:

40 not found

Why stop at index 2?

Because if 40 had been inserted, it would have used index 2 before going farther.

17. Search Dry Run Table

Search key	Initial hash	Search path	Result
45	5	Index 5 contains 45	Found immediately
74	4	Check 43, 45, 26, 25, 74	Found after 5 checks
40	0	Check 30, 29, then empty index 2	Not found

18. Loading Factor

18.1 What loading factor means

The loading factor tells us how full the hash table is.

Formula:

$$\lambda = n / \text{size}$$

Where:

Symbol	Meaning
<code>lambda</code>	Loading factor
<code>n</code>	Number of stored keys
<code>size</code>	Table size

Example:

```
n = 5
size = 10
lambda = 5 / 10 = 0.5
```

This means the table is half full.

18.2 Loading factor rule for linear probing

For linear probing, the recommended rule is:

$$\lambda \leq 0.5$$

Simple meaning:

Keep the table at most half full.

If the table size is `10`, try to store at most about `5` keys.

18.3 Why lambda should be small

Linear probing needs empty spaces to work well.

If the table becomes too full:

Collisions become more common.
There are fewer empty slots.
Probe sequences become longer.
Search becomes slower.
Insertion becomes slower.
Clusters become larger.

So linear probing usually needs more empty space than chaining.

18.4 Example of loading factor

If:

Table size = 10
Number of keys = 5

Then:

$$\text{lambda} = 5 / 10 = 0.5$$

This is acceptable for linear probing.

But if:

Table size = 10
Number of keys = 9

Then:

$$\text{lambda} = 9 / 10 = 0.9$$

This is too high for basic linear probing.

At 0.9, the table is almost full, and searching may become slow.

19. Average Search Formulas from the Curriculum

The curriculum lists formulas for average search cost in linear probing.

Operation	Formula	Beginner meaning
Successful search	$(1 / \lambda) * \ln(1 / (1 - \lambda))$	As the table gets fuller, successful search needs more probes on average
Unsuccessful search	$1 / (1 - \lambda)$	Missing keys become expensive to search for when the table is almost full

You do not need to derive these formulas as a beginner.

The important idea is:

When λ increases, searching gets slower.

Especially for unsuccessful search, if λ is close to 1, then:

$1 / (1 - \lambda)$

becomes very large.

Example:

$\lambda = 0.9$
 $1 / (1 - 0.9) = 1 / 0.1 = 10$

So unsuccessful search may need many checks.

20. Primary Clustering

20.1 What primary clustering means

Primary clustering means keys form long continuous groups in the hash table.

Example:

```
Index 3: 23
Index 4: 43
Index 5: 45
Index 6: 26
Index 7: 25
Index 8: 74
Index 9: 19
```

This is a cluster because many filled positions are next to each other.

20.2 Why primary clustering is bad

If a new key hashes to any place inside the cluster, it must walk through many full slots.

Example:

If a new key hashes to index `4`, it may need to check:

```
4, 5, 6, 7, 8, 9, 0, 1, ...
```

This makes insertion and search slower.

20.3 Traffic jam analogy

Primary clustering is like a traffic jam.

Once a traffic jam starts, new cars join it and make it longer.

In linear probing:

```
Once a cluster starts, new keys often join it and make it bigger.
```

Simple meaning:

```
Linear probing can create crowded blocks that keep growing.
```

21. Deletion in Linear Probing

Deletion is one of the most important problems in linear probing.

In chaining, deletion is simple:

Find the node and unlink it from the linked list.

But in linear probing, deletion can break future searches.

22. Why Simple Deletion Is Wrong

Suppose the table contains:

Index 4: 43
Index 5: 45
Index 6: 26
Index 7: 25
Index 8: 74

Search for 74 starts at:

$74 \bmod 10 = 4$

Search path:

Index 4 → Index 5 → Index 6 → Index 7 → Index 8

Now suppose we delete 45 from index 5 and make index 5 truly empty.

The table becomes:

Index 4: 43
Index 5: empty
Index 6: 26
Index 7: 25
Index 8: 74

Now search for 74 again:

Index 4: 43, not 74
Index 5: empty
Stop

The algorithm incorrectly says:

74 not found

But 74 is still at index 8.

So the problem is:

If we clear a slot, we may break the search path for keys after it.

23. Rehashing Solution

One solution is **rehashing**.

Rehashing means after deletion, we rebuild part of the table.

Basic idea:

1. Delete the key.
2. Take the affected keys after it.
3. Remove them temporarily.
4. Insert them again using the hash function.

This makes the table correct again.

But it can be expensive.

Why?

Because many keys may need to be reinserted.

Simple meaning:

Rehashing is correct, but it can be slow.

24. Tombstone Solution

Another solution is to use a **tombstone**.

A tombstone is a special marker that means:

This slot used to contain a key, but that key was deleted.

Instead of making the slot truly empty, we mark it as:

DELETED

24.1 Difference between EMPTY and DELETED

Slot state	Meaning	What search should do
EMPTY	This slot has never been used	Stop searching
DELETED	This slot had a key before, but it was removed	Continue searching
Filled	This slot contains a real key	Compare with search key

This is very important.

Search should stop at **EMPTY**, but not at **DELETED**.

24.2 Tombstone example

Before deletion:

Index 4: 43
Index 5: 45
Index 6: 26
Index 7: 25
Index 8: 74

Delete **45**.

Instead of:

Index 5: empty

Use:

Index 5: DELETED

Now search for 74 :

Index 4: 43, not 74
Index 5: DELETED, continue
Index 6: 26, not 74
Index 7: 25, not 74
Index 8: 74, found

So the tombstone keeps the search path alive.

24.3 Insertion with tombstones

A deleted slot can be reused for insertion.

Example:

Index 5: DELETED

If a new key is inserted and its probe path reaches index 5, the algorithm may reuse index 5.

So:

DELETED means available for insertion, but search must continue past it.

25. Important Rule for Search with Tombstones

Search rules:

If slot contains the key:
 Found
If slot is DELETED:

```
Continue searching
If slot is EMPTY:
    Stop and say not found
```

Why?

Because **DELETED** means there may be keys later in the same probe sequence.

But **EMPTY** means no key has ever been placed beyond this point in that probe path.

26. C Code Example from the Curriculum Idea

The basic C code idea uses:

```
#define SIZE 10

int hash(int key) {
    return key % SIZE;
}
```

The hash function returns the starting index.

Example:

```
hash(35) = 35 mod 10 = 5
```

If index **5** is full, probing checks the next index.

27. Basic Probe Function

A basic probe function can look like this:

```
int probe(int H[], int key) {
    int index = hash(key);
    int i = 0;

    while (H[(index + i) % SIZE] != 0) {
        i++;
    }
}
```

```
    return (index + i) % SIZE;
}
```

Meaning:

Start at the hash index.
Keep moving forward while the slot is not empty.
Use mod SIZE for wrap-around.
Return the first empty slot found.

Important limitation:

This code uses `0` to mean empty.

So this version cannot safely store key `0`.

28. Complete C Program with Tombstones

This program uses:

```
EMPTY = 0
DELETED = -1
```

Important warning:

This means the program assumes real keys are not `0` or `-1`.

```
#include <stdio.h>

#define SIZE 10
#define EMPTY 0
#define DELETED -1

int hash(int key) {
    return key % SIZE;
}

void printTable(int H[]) {
    for (int i = 0; i < SIZE; i++) {
        printf("%d: ", i);

        if (H[i] == EMPTY) {
```

```

        printf("EMPTY");
    } else if (H[i] == DELETED) {
        printf("DELETED");
    } else {
        printf("%d", H[i]);
    }

    printf("\n");
}
}

int search(int H[], int key) {
    int index = hash(key);

    for (int i = 0; i < SIZE; i++) {
        int probeIndex = (index + i) % SIZE;

        if (H[probeIndex] == EMPTY) {
            return -1;
        }

        if (H[probeIndex] == key) {
            return probeIndex;
        }
    }

    return -1;
}

int insert(int H[], int key) {
    int index = hash(key);
    int firstDeleted = -1;

    for (int i = 0; i < SIZE; i++) {
        int probeIndex = (index + i) % SIZE;

        if (H[probeIndex] == key) {
            return probeIndex;
        }

        if (H[probeIndex] == DELETED && firstDeleted == -1) {
            firstDeleted = probeIndex;
        }

        if (H[probeIndex] == EMPTY) {
            if (firstDeleted != -1) {
                probeIndex = firstDeleted;
            }
        }
    }
}

```

```

        H[probeIndex] = key;
        return probeIndex;
    }
}

if (firstDeleted != -1) {
    H[firstDeleted] = key;
    return firstDeleted;
}

return -1; // table full
}

void deleteKey(int H[], int key) {
    int index = search(H, key);

    if (index != -1) {
        H[index] = DELETED;
    }
}

int main() {
    int HT[SIZE] = {EMPTY};

    int keys[] = {26, 30, 45, 23, 25, 43, 74, 19, 29};
    int n = sizeof(keys) / sizeof(keys[0]);

    for (int i = 0; i < n; i++) {
        int pos = insert(HT, keys[i]);
        printf("Inserted %d at index %d\n", keys[i], pos);
    }

    printf("\nFinal table:\n");
    printTable(HT);

    int key = 74;
    int found = search(HT, key);

    if (found == -1) {
        printf("\nSearch %d: not found\n", key);
    } else {
        printf("\nSearch %d: found at index %d\n", key, found);
    }

    deleteKey(HT, 45);

    printf("\nAfter tombstone deletion of 45:\n");

```

```
printTable(HT);

found = search(HT, 74);

if (found == -1) {
    printf("\nSearch 74 after deleting 45: not found\n");
} else {
    printf("\nSearch 74 after deleting 45: found at index %d\n", found);
}

return 0;
}
```

29. Code Explanation: Constants

```
#define SIZE 10
#define EMPTY 0
#define DELETED -1
```

Meaning:

Constant	Meaning
<code>SIZE</code>	Table has 10 slots
<code>EMPTY</code>	Slot is empty
<code>DELETED</code>	Slot had a key but it was deleted

Important:

If `0` is a valid key, using `0` as `EMPTY` is dangerous.

If `-1` is a valid key, using `-1` as `DELETED` is dangerous.

A safer real-world design uses a separate status array.

30. Code Explanation: Hash Function

```
int hash(int key) {  
    return key % SIZE;  
}
```

This converts the key into the original index.

Examples:

```
hash(26) = 26 mod 10 = 6  
hash(45) = 45 mod 10 = 5  
hash(74) = 74 mod 10 = 4
```

31. Code Explanation: Search Function

```
int search(int H[], int key) {  
    int index = hash(key);  
  
    for (int i = 0; i < SIZE; i++) {  
        int probeIndex = (index + i) % SIZE;  
  
        if (H[probeIndex] == EMPTY) {  
            return -1;  
        }  
  
        if (H[probeIndex] == key) {  
            return probeIndex;  
        }  
    }  
  
    return -1;  
}
```

Important line:

```
int probeIndex = (index + i) % SIZE;
```

This calculates the next position to check.

The search stops only when it finds `EMPTY`.

It does not stop at `DELETED` because there may be keys after a deleted slot.

32. Code Explanation: Insert Function

```
int insert(int H[], int key)
```

This function inserts a key using linear probing.

Important idea:

```
Try the original index first.  
If it is full, keep moving forward.
```

The variable:

```
int firstDeleted = -1;
```

remembers the first deleted slot found.

Why?

Because deleted slots can be reused for insertion.

But the function still keeps searching to make sure the key is not already in the table.

33. Code Explanation: Delete Function

```
void deleteKey(int H[], int key) {  
    int index = search(H, key);  
  
    if (index != -1) {  
        H[index] = DELETED;  
    }  
}
```

This function does not make the slot empty.

It marks the slot as:

DELETED

Why?

Because making it empty could break future searches.

34. Program Dry Run Summary

The program inserts:

26, 30, 45, 23, 25, 43, 74, 19, 29

Final table:

0: 30
1: 29
2: EMPTY
3: 23
4: 43
5: 45
6: 26
7: 25
8: 74
9: 19

Then it searches for 74 .

$74 \bmod 10 = 4$

Search path:

4 → 5 → 6 → 7 → 8

It finds 74 at index 8 .

Then it deletes 45 .

Index 5 becomes:

DELETED

Now search for 74 again:

4 → 5(DELETED) → 6 → 7 → 8

It still finds 74.

This proves why tombstones are useful.

35. Time Complexity

Linear probing tries to achieve average $O(1)$ insertion and search.

But this depends strongly on the loading factor.

Situation	Performance
Table has many empty slots	Fast
Table is half full or less	Usually acceptable
Table is almost full	Slow
Many clusters exist	Slow

Worst case:

$O(N)$

Why?

Because in the worst case, the algorithm may need to scan many positions.

36. Linear Probing vs Chaining

Feature	Chaining	Linear Probing
Collision handling	Linked list at bucket	Move to next empty slot

Feature	Chaining	Linear Probing
Extra memory	Uses linked-list nodes	No linked lists
Deletion	Easier	Difficult
Search path	One chain	Probe sequence inside array
Main problem	Long chains	Primary clustering
Loading factor	Can be greater than 1	Should be kept around <code><= 0.5</code>

Simple comparison:

Chaining spreads collisions vertically using lists.
 Linear probing spreads collisions horizontally inside the array.

37. Common Beginner Mistakes

Mistake 1: Confusing open hashing and open addressing

Open hashing means chaining.

Open addressing means closed hashing.

This sounds confusing, but remember:

Open hashing = use outside linked-list memory.
 Open addressing = find another open address inside the table.

Mistake 2: Forgetting wrap-around

Wrong idea:

After index 9, stop.

Correct idea:

After index 9, go back to index 0.

Use:

```
(index + i) mod size
```

Mistake 3: Starting `i` from 1

In probing, `i` should start at `0`.

Why?

Because the first try should be the original hash index.

```
i = 0 → original index  
i = 1 → next index
```

Mistake 4: Stopping at a deleted slot

Wrong:

```
If slot is DELETED, stop.
```

Correct:

```
If slot is DELETED, continue searching.
```

Search should only stop at `EMPTY`.

Mistake 5: Clearing a deleted slot to empty

Wrong:

```
Delete key by making the slot EMPTY.
```

Correct:

```
Delete key by marking the slot DELETED.
```

This avoids breaking the search path.

Mistake 6: Overfilling the table

Linear probing needs empty spaces.

If the table is too full, probing becomes slow.

Remember:

```
lambda <= 0.5
```

Mistake 7: Ignoring primary clustering

A cluster is a long block of filled slots.

Clusters make future insertions and searches slower.

Linear probing is simple, but primary clustering is its major weakness.

Mistake 8: Using 0 as empty when 0 is a valid key

If 0 can be stored as a real key, then this is dangerous:

```
#define EMPTY 0
```

Because the program cannot know whether 0 means:

```
real key 0
```

or:

```
empty slot
```

A better solution is to use a separate status array.

38. Very Important Formulas

Original hash function:

$$H(x) = x \bmod \text{size}$$

Linear probing formula:

$$H'(x) = (H(x) + i) \bmod \text{size}$$

For linear probing:

$$i = 0, 1, 2, 3, \dots$$

Loading factor:

$$\text{lambda} = n / \text{size}$$

Recommended loading factor:

$$\text{lambda} \leq 0.5$$

Average successful search formula from the curriculum:

$$(1 / \text{lambda}) * \ln(1 / (1 - \text{lambda}))$$

Average unsuccessful search formula from the curriculum:

$$1 / (1 - \text{lambda})$$

39. Phase 3 Quick Revision Sheet

Concept	Simple meaning
Closed hashing	Store all keys inside the original array
Open addressing	Find another open address when collision happens
Linear probing	Move one slot forward at a time
Collision	Two keys want the same index
Probe	A position checked after collision
Wrap-around	After the last index, continue at index 0
Stop-at-empty rule	If search reaches empty, key is not present
Loading factor	How full the table is
$\lambda = n / \text{size}$	Formula for loading factor
$\lambda \leq 0.5$	Recommended rule for linear probing
Primary clustering	Long continuous blocks of filled slots
Deletion problem	Clearing a slot can break search
Rehashing	Reinsert affected keys after deletion
Tombstone	Mark deleted slot as DELETED, not EMPTY

40. Phase 3 Full Summary

Phase 3 teaches **closed hashing with linear probing**.

In Phase 1, you learned that hashing converts a key into an index.

In Phase 2, you learned that chaining solves collisions using linked lists.

In Phase 3, you learn a different approach.

In closed hashing, all keys stay inside the original array.

If a key hashes to a full slot, linear probing checks the next slot, then the next, and continues until it finds an empty slot.

The formula is:

$$H'(x) = (H(x) + i) \bmod \text{size}$$

where:

$$i = 0, 1, 2, 3, \dots$$

The `mod size` part gives wrap-around.

Searching follows the same probe path as insertion.

If search reaches an empty slot, it stops because the key cannot be after that empty slot.

Linear probing works best when the table is not too full.

The loading factor is:

$$\lambda = n / \text{size}$$

For linear probing, it is recommended to keep:

$$\lambda \leq 0.5$$

The main weakness of linear probing is **primary clustering**.

Primary clustering means keys form long continuous blocks.

These blocks make future insertions and searches slower.

Deletion is also difficult.

If we simply make a deleted slot empty, future searches may stop too early.

There are two solutions:

1. Rehashing
2. Tombstones

Rehashing rebuilds the affected part of the table, but it can be slow.

Tombstones mark deleted slots as `DELETED`, so search continues past them.

The most important idea is:

In linear probing, collision means move forward inside the same array.

41. What You Should Be Able to Explain After Phase 3

After studying Phase 3, you should be able to explain:

1. What closed hashing means
2. What open addressing means
3. Why linear probing does not use linked lists
4. How linear probing handles collisions
5. The formula $H'(x) = (H(x) + i) \bmod \text{size}$
6. Why i starts at 0
7. Why modulus gives wrap-around
8. How to insert keys using linear probing
9. How to search for keys using linear probing
10. Why search stops at an empty slot
11. What loading factor means
12. How to calculate $\lambda = n / \text{size}$
13. Why linear probing should keep $\lambda \leq 0.5$
14. What primary clustering means
15. Why primary clustering makes linear probing slower
16. Why deletion is difficult in linear probing
17. Why clearing a slot can break search
18. What rehashing means
19. What tombstones mean
20. Why search continues past DELETED slots
21. Why search stops at EMPTY slots
22. Common mistakes in linear probing
23. Why Phase 4 is needed after Phase 3

42. Mini Practice 1

Use:

Table size = 10
 $H(x) = x \bmod 10$
Keys = 12, 22, 32

Calculations:

```
12 mod 10 = 2
22 mod 10 = 2
32 mod 10 = 2
```

Insertion:

```
12 goes to index 2
22 wants index 2, but index 2 is full, so it goes to index 3
32 wants index 2, but index 2 and 3 are full, so it goes to index 4
```

Final table:

```
Index 0: empty
Index 1: empty
Index 2: 12
Index 3: 22
Index 4: 32
Index 5: empty
Index 6: empty
Index 7: empty
Index 8: empty
Index 9: empty
```

43. Mini Practice 2

Use:

```
Table size = 10
H(x) = x mod 10
Keys = 18, 28, 38, 48
```

Calculations:

```
18 mod 10 = 8
28 mod 10 = 8
38 mod 10 = 8
48 mod 10 = 8
```

Insertion:

18 goes to index 8
28 wants index 8, but it is full, so it goes to index 9
38 wants index 8, but 8 and 9 are full, so wrap-around happens and it goes to index 0
48 wants index 8, but 8, 9, and 0 are full, so it goes to index 1

Final table:

Index 0: 38
Index 1: 48
Index 2: empty
Index 3: empty
Index 4: empty
Index 5: empty
Index 6: empty
Index 7: empty
Index 8: 18
Index 9: 28

Main lesson:

Wrap-around allows probing to continue from the start of the table.

44. Mini Practice 3: Search

Use this table:

Index 0: 38
Index 1: 48
Index 2: empty
Index 3: empty
Index 4: empty
Index 5: empty
Index 6: empty
Index 7: empty
Index 8: 18
Index 9: 28

Search for 48:

```
48 mod 10 = 8
Index 8: 18, not 48
Index 9: 28, not 48
Index 0: 38, not 48
Index 1: 48, found
```

Search for 58:

```
58 mod 10 = 8
Index 8: 18, not 58
Index 9: 28, not 58
Index 0: 38, not 58
Index 1: 48, not 58
Index 2: empty, stop
```

Result:

58 not found

45. Final Simple Explanation

Linear probing is a way to solve collisions without linked lists.

If a key wants an index that is already full, it checks the next index.

If that is also full, it checks the next one again.

The formula is:

$$H'(x) = (H(x) + i) \bmod \text{size}$$

The `mod size` part makes the search wrap around from the end of the table to the beginning.

Linear probing is simple, but it has two big problems:

1. Primary clustering
2. Difficult deletion

Primary clustering means filled slots form long blocks.

Deletion is difficult because making a slot empty can break future searches.

To solve deletion, we can use:

Tombstones

A tombstone means:

This slot was deleted, but search should continue past it.

The heart of Phase 3 is:

Collision does not create a chain.
Collision makes us move forward inside the same table.

Phase 4 comes next because it improves the probing idea.

Instead of always moving one step at a time, Phase 4 uses:

Quadratic probing
Double hashing

These methods reduce the clustering problem of linear probing.